ОТЧЁТ

по лабораторной работе №2

по дисциплине «Операционные системы»

Вариант 7

Выполнил: Пьянзин Михаил Андреевич М80-201БВ-25

1. Цель работы

Приобретение практических навыков в:

- управлении процессами в операционной системе;
- обеспечении обмена данными между процессами посредством каналов (pipe).

2. Постановка задачи

Вариант 7 (группа вариантов 2). Родительский процесс запрашивает у пользователя имя файла и открывает его на чтение. Стандартный ввод дочернего процесса перенаправляется на этот файл, стандартный вывод - в канал (pipe1). Дочерний процесс читает строки вида «число число число», суммирует числа типа float в каждой строке и печатает результат в свой стандартный вывод. Родительский процесс читает результат из pipe1 и выводит его в свой стандартный вывод.

3. Архитектура решения

Схема взаимодействия процессов:

```
[Файл]  - (dup2, stdin) ->  [CHILD]  -(dup2, stdout) ->  [pipe1: write]
                                                               |
                                                         [pipe1: read]
                                                               |
                                                         [PARENT]  -> экран
```

3.1. Используемые системные вызовы

- open() - открытие файла на чтение (O_RDONLY);
- pipe() - создание неименованного канала;
- fork() - создание дочернего процесса;
- dup2() - перенаправление стандартных потоков ввода/вывода;
- execl() - запуск программы child в дочернем процессе;
- close() - закрытие неиспользуемых дескрипторов;
- read() / write() - чтение и запись данных через канал;
- waitpid() - ожидание завершения дочернего процесса.

4. Листинг ключевых фрагментов кода

4.1. parent.cpp - создание канала и процесса

```
int file_fd = open(file_name.c_str(), O_RDONLY);
   if (file_fd == -1) {
       perror("open");
       return 1;
   }
   int pipe1[2];
   int result = pipe(pipe1);
   if (result == -1) {
       perror("pipe");
       return 1;
   }
```

4.2. parent.cpp - ветка дочернего процесса

```
} else if (pid == 0) {
        std::cout << "Child PID: " << getpid()<< std::endl;
        if (dup2(file_fd, 0) == -1) {
            perror("dup2 stdin");
            return 1;
        }
        if (dup2(pipe1[1], 1) == -1) {
            perror("dup2 stdout");
            return 1;
        }
        close(file_fd);
        close(pipe1[0]);
        close(pipe1[1]);
        execl("./child", "child", (char*)NULL);
        perror("execl");
        return 1;
}
```

4.3. parent.cpp - ветка родительского процесса

```
 } else {
        std::cout << "Parent PID: " << getpid()<< std::endl;
        close(pipe1[1]);
        close(file_fd);
        char buf[256];
        ssize_t n;
        while ((n = read(pipe1[0], buf, 256)) > 0){
            write(1, buf, n);
        }
        if (n == -1) {
            perror("read");
        }
        close(pipe1[0]);
        int status;
        waitpid(pid, &status, 0);
    }
}
```

4.4. child.cpp - чтение и суммирование

```
    char *line = NULL;
    size_t cap = 0;
    while (getline(&line, &cap, stdin) != -1)
    {
        char *ptr = line;
        char *end;
        float num;
        float sum = 0;
        int error = 0;

        while (1)
        {
            num = strtof(ptr, &end);
            if (end == ptr)
            {
                break;
            }
            sum += num;
            ptr = end;
```

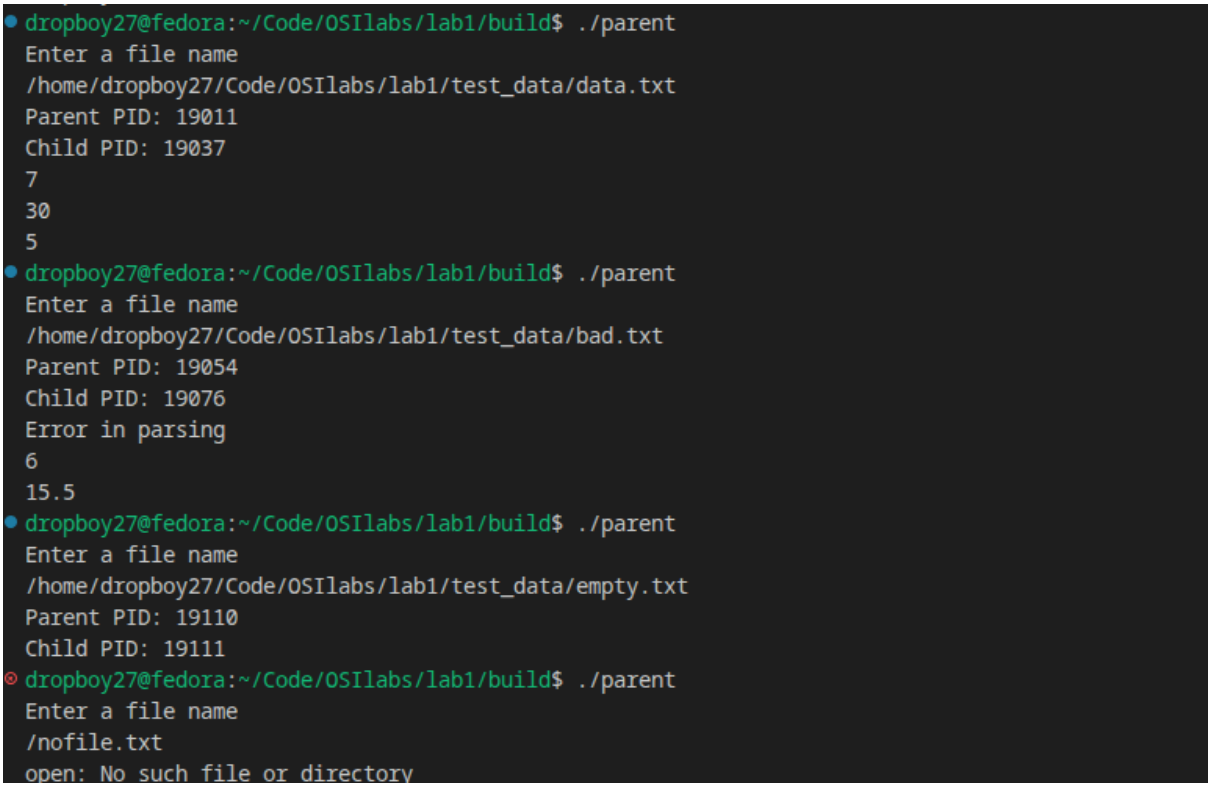
```
        }

        while (*ptr == ' ' || *ptr == '\n' || *ptr == '\t')
        {
            ptr++;
        }

        if (*ptr == '\0')
        {
            printf("%g\n", sum);
        }
        else
        {
            fprintf(stderr, "Error in parsing\n");
        }
    }
    free(line);
    return 0;
}
```

5. Результаты работы

5.1. Базовые сценарии



5.2. Проверка обработки ошибок

Проверены следующие граничные случаи:

- несуществующий файл - корректный вывод сообщения об ошибке через perror(), без падения программы;
- пустой файл - программа завершается корректно, без вывода и без зависания;

- строка с некорректными данными - выводится сообщение об ошибке разбора, обработка следующих строк продолжается.

5.3. Проверка отсутствия зомби-процессов

После завершения программы проверено отсутствие зомби-процессов командой ps.

```
dropboy27@fedora:~/Code/OSIlabs/lab1/build$ ps -eo pid,stat,cmd | grep -w Z
  19598 S+   grep --color=auto -w Z
```

5.4. Трассировка системных вызовов (strace)

Для подтверждения корректной работы архитектуры программа была запущена под strace -f.

build > trace.log

```
12952 execve("./parent", ["./parent"], 0x7fff1d3b4438 /* 82 vars */) = 0
12952 brk(NULL)                           = 0x1ad47000
12952 access("/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or directory)
12952 openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
12952 fstat(3, {st_mode=S_IFREG|0644, st_size=98191, ...}) = 0
12952 mmap(NULL, 98191, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7fa5eff1b000
12952 close(3)                            = 0
12952 openat(AT_FDCWD, "/lib64/libstdc++.so.6", O_RDONLY|O_CLOEXEC) = 3
12952 read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0"..., 832) = 832
12952 fstat(3, {st_mode=S_IFREG|0755, st_size=2884696, ...}) = 0
12952 mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7fa5eff19000
12952 mmap(NULL, 2822848, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7fa5efc00000
12952 mmap(0x7fa5efd71000, 1224704, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x171000) = 0x7fa5efd71000
12952 mmap(0x7fa5efe9c000, 73728, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x29c000) = 0x7fa5efe9
12952 mmap(0x7fa5efeae000, 12992, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7fa5efeae000
12952 close(3)                            = 0
12952 openat(AT_FDCWD, "/lib64/libm.so.6", O_RDONLY|O_CLOEXEC) = 3
12952 read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0"..., 832) = 832
12952 fstat(3, {st_mode=S_IFREG|0755, st_size=1153848, ...}) = 0
12952 mmap(NULL, 1138704, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7fa5efae9000
12952 mmap(0x7fa5efb78000, 548864, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x8f000) = 0x7fa5efb78000
12952 mmap(0x7fa5efbfe000, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x114000) = 0x7fa5efbfe
12952 close(3)                            = 0
12952 openat(AT_FDCWD, "/lib64/libgcc_s.so.1", O_RDONLY|O_CLOEXEC) = 3
12952 read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0"..., 832) = 832
12952 fstat(3, {st_mode=S_IFREG|0755, st_size=186512, ...}) = 0
12952 mmap(NULL, 180680, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7fa5efeec000
12952 mmap(0x7fa5eff10000, 28672, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x24000) = 0x7fa5eff10000
12952 mmap(0x7fa5eff17000, 4096, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x2b000) = 0x7fa5eff170
12952 mmap(0x7fa5eff18000, 456, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7fa5eff18000
12952 close(3)                            = 0
12952 openat(AT_FDCWD, "/lib64/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
12952 read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\2408\0\0\0\0\0\0"..., 832) = 832
12952 pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0"..., 784, 64) = 784
12952 fstat(3, {st_mode=S_IFREG|0755, st_size=2482096, ...}) = 0
12952 pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0"..., 784, 64) = 784
12952 mmap(NULL, 2067312, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7fa5ef8f0000
```

6. Выводы

В ходе выполнения лабораторной работы были получены практические навыки работы с процессами и межпроцессным взаимодействием в ОС Linux.

Освоены следующие системные примитивы:

- fork() - создание дочернего процесса как копии родительского;
- pipe() - создание неименованного канала для передачи данных между процессами;
- dup2() - перенаправление стандартных потоков ввода/вывода на файл и на канал;
- exec() (execl) - замена образа процесса на другую программу;
- waitpid() - ожидание завершения дочернего процесса и предотвращение появления зомби-процессов.

Реализована программа из двух независимых исполняемых файлов (parent и child), взаимодействующих через файл и pipe: родительский процесс открывает файл, перенаправляет стандартный ввод дочернего процесса на этот файл, а стандартный вывод - в канал, дочерний процесс суммирует числа в каждой строке и передаёт результат обратно родителю, который выводит его на экран.

В ходе выполнения работы возникли несоклько проблем: создание работа с дескрипторами на pipe в дочернем процессе, буферизация вывода при перенаправлении в pipe, которая была решенеа при помощи unitbuf.

Были протестированны граничные случаи(пустой файл, несуществующий файл, некорректные данные) и проверена корректное выполенние программы без зомби процессов.

Так же я получил опыт работы с strace — он помогу увидеть результат работы системныъх вызовов и понять как они соотносятся с кодом